[[exhaustive]] attribute for enums

- Document number: P0375R0
- Date: 2016-05-29

- Reply-to: David Sankel <david@stellarscience.com>

- Audience: Evolution

Abstract

Can a compiler confidently elide warnings on `switch` statements that cover all enumerators of an `enum`? Due the many ways `enum` is used, it turns out the answer is no. Compiler vendors choose either to be overly pessimistic, and always warn when default labels are missing, or overly optimistic, and omit warnings when they're needed.

The compiler needs a little more information on how a particular `enum` is intended to be used. This proposal makes that possible. The proposed [[exhaustive]] attribute will result in better compiler warnings and less bugs.


```
enum [[exhaustive]] color {
  red,
  green,
  blue
};
```

Motivation

Given the following enum definition and instance,

```
enum color {
  red,
  green,
  blue
};

color c = /* etc. */;
```

, we would like compilers to emit a warning here,

```
switch(c) {
case red:
  std::cout << "red" << std::endl;
  break;
case green:
  std::cout << "green" << std::endl;
  break;

  // Hey, where's blue?
};
```

, but not here,

```
switch(c) {
case red:
  std::cout << "red" << std::endl;
  break;
case green:
  std::cout << "green" << std::endl;
  break;
case blue:
  std::cout << "blue" << std::endl;
  break;
};

// Perfect, all bases are covered.
```

. The latter `switch` statement is safe in that it handles all the possible enumerators.

Unfortunately, other `enum` uses prevent the compiler from always knowing when a `switch` is safe. Consider the following example:

```
enum color_channels
{
  red = 1 << 0,
  green = 1 << 1,
  blue = 1 << 2,
  alpha = 1 << 3
};

color_channels cc = /* etc. */;
```

In this case, the enumeration is intended to be used as any combination of a red, green, blue, and alpha channel. This switch,

```
switch(cc) {
case red:
  std::cout << "red" << std::endl;
  break;
case green:
  std::cout << "green" << std::endl;
  break;
case blue:
  std::cout << "blue" << std::endl;
  break;
case alpha:
  std::cout << "alpha" << std::endl;
  break;
};
```

, *should* produce a warning because it fails to handle anything other than *only* the red, green, or blue channel.

To make matters worse, there is yet another common enumeration pattern:

```
enum color_selections
{
  red,
  green,
  blue,
  hot_pink,
  user_0
};

color_selection cs = /* etc. */;
```

In this case, the intent is that the `user_0` enumerator marks the beginning of an extension point where an arbitrary number of colors can be specified. The following snippet,

```
switch(cs) {
case red:
  std::cout << "red" << std::endl;
  break;
case green:
  std::cout << "green" << std::endl;
  break;
case blue:
  std::cout << "blue" << std::endl;
  break;
case hot_pink:
  std::cout << "hot pink" << std::endl;
  break;
case user0:
  std::cout << extra_color_names[0] << std::endl;
  break;
};
```

, again *should* produce a warning since only the 0th user color is handled.

All of `color`, `color_channels`, and `color_selection` are common uses of enumerators, but only the first should elide `switch` warnings when all the enumerators are handled. Because the compiler cannot consistently divine the intent of the developer, we propose a `[[exhaustive]]` attribute on enumerations that indicates that the enumerators represent *all* the valid values for that enumeration.

```
enum [[exhaustive]] color {
  red,
  green,
  blue
};
```

Wording

Add new section to [dcl.attr] 7.6:

7.6.9 Exhaustive attribute [dcl.attr.exhaustive]

1. The *attribute-token* `exhaustive` may be applied to the *declarator-id* in the declaration of an enumeration. It shall appear at most once in each *attribute-list* and no *attributeargument-clause* shall be present.

2. [*Note:* For an enumeration marked `exhaustive`, implementations are encouraged not to emit a warning on `switch` statements with cases covering all enumerators of that enum. — *end note*]

3. [*Example:*

   ```
   enum [[exhaustive]] color {
     red,
     green,
     blue
   };
   ```

   — *end example*]

Conclusion

The `[[exhaustive]]` attribute allows developers to clarify the intent of their enumerations in a way fixes a longstanding issue with `switch` compiler warnings that are either spurious or dangerously absent.